

Lab 5

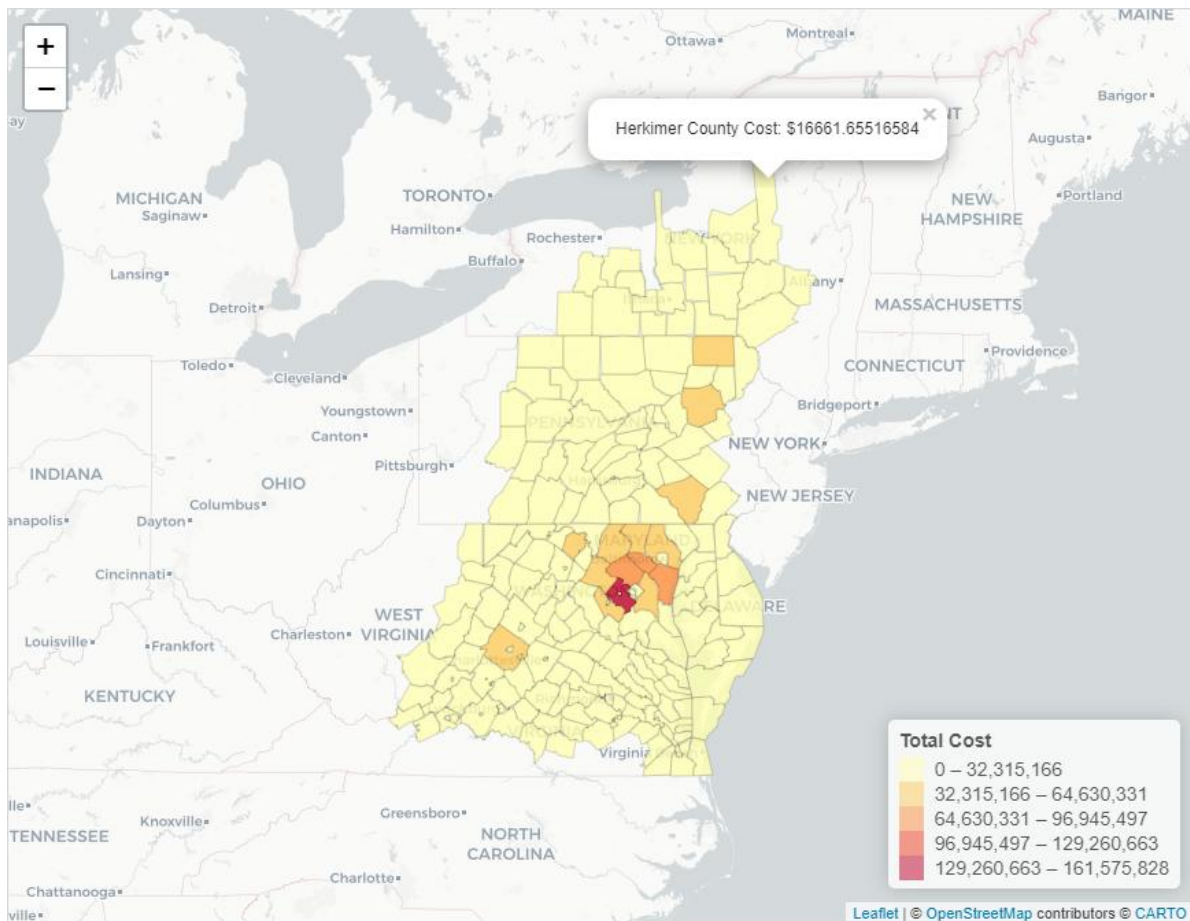
Environmental Analysis in R

Srishti Roy Chowdhury

Task-1:

```
> #data connection
> bmps <- read_csv("./data/BMPPreort2016_landbmps.csv")
Rows: 69601 Columns: 18
# Column specification
Delimiter: ","
chr (11): StateAbbreviation, GeographyName, Geography, Agency, BMPShortName, BMP, BMPType, Unit, Sector, FromLoadSour...
dbl (7): AmountSubmitted, AmountBackedOut, AmountNotBackedOut, AmountCredited, Excess, TotalAmountCredited, Cost

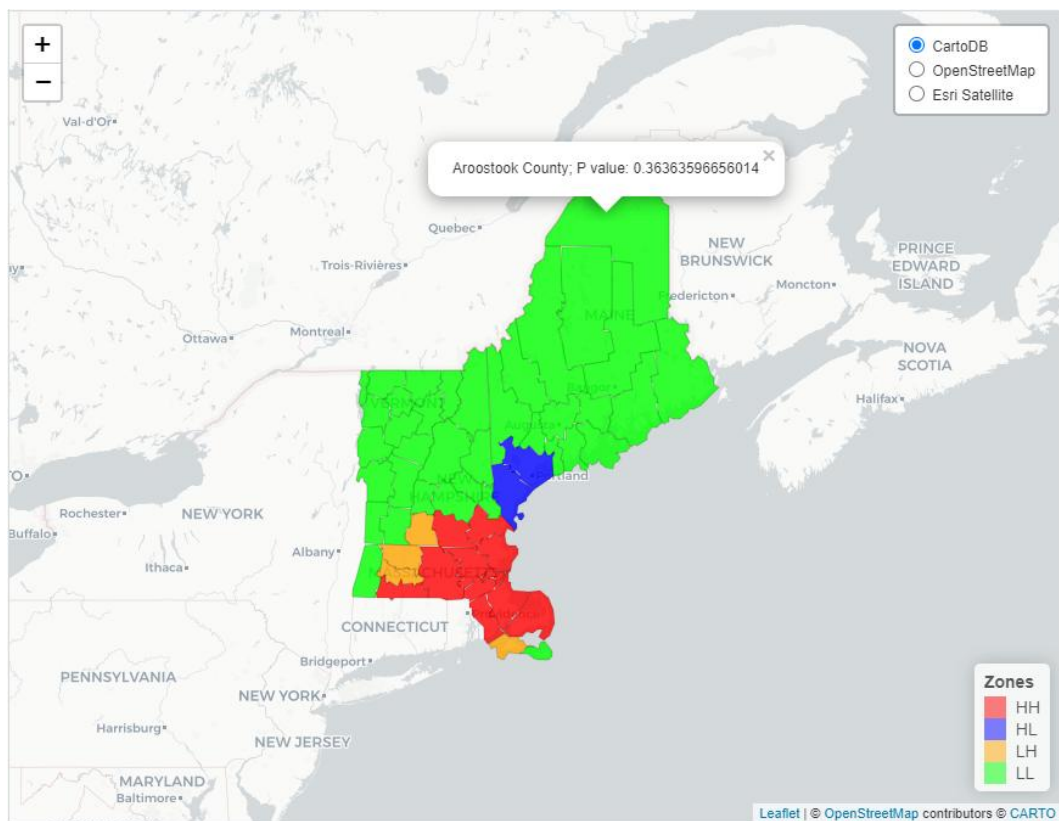
i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
> counties <- sf::read_sf("./data/County_Boundaries.shp")
>
> #data organization
> bmps <- bmps %>% mutate(, geoid = stringr::str_sub(GeographyName, 1, 5))
> counties.bmps <- left_join(counties, bmps, by = c("GEOID10" = "geoid"))
> counties_bmps <- counties.bmps %>%
+   st_drop_geometry() %>% #drop the geometry to reduce the running time of the code
+   group_by(GEOID10) %>% summarise(TotalCost = sum(Cost, na.rm = TRUE)) %>%
+   left_join(counties, by = c("GEOID10" = "GEOID10")) %>%
+   st_as_sf()
>
> #creating dynamic map
> breaks <- classIntervals(counties_bmps$TotalCost, n = 5, style = "equal")$brks
> colors <- brewer.pal(5, "YlOrRd")
> pal <- colorBin(palette = colors, domain = counties_bmps$TotalCost, bins = breaks)
> leaflet(data = counties_bmps) %>%
+   addProviderTiles(providers$CartoDB.Positron) %>%
+   addPolygons(fillColor = ~pal(TotalCost), stroke = TRUE, color = "black", weight = 0.3, fillOpacity = 0.8, popup = ~paste
0(NAME10," County Cost: $", TotalCost)) %>%
+   addLegend(pal = pal, values = ~TotalCost, title = "Total Cost", position = "bottomright")
```



Task-2:

```
> #data connection
> states4 <- sf::read_sf("./data/states4.shp")
> age.sex <- read_csv("./data/age.sex.csv")
New names:
* `` -> `...1`
Rows: 3221 Columns: 720
Column specification
Delimiter: ","
chr (1): GEOID
dbl (719): ...1, B01001e1, B01001m1, B01001e2, B01001m2, B01001e3, B01001m3, B01001e4, B01001m4, B01001e5, B01001m5, ...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
>
> #data organization
> age.sex <- age.sex %>% mutate(.fixed_geoid = str_sub(GEOID, start = 8, end = -1))
> states4 <- states4 %>% left_join(., age.sex, by = c("GEOID" = "fixed_geoid"))
> states4.projected <- states4 %>% sf::st_transform(., crs=4326)
>
> #Creating Moran plot (really, "LISA") quadrants
> neighbor <- spdep::poly2nb(states4.projected, queen = TRUE) #Calculate Queen's case neighbors
> lw <- nb2listw(neighbor, style="w", zero.policy=TRUE)
> F.lag <- lag.listw(lw, states4.projected$B01001e26)
> #moran.test(states4.projected$B01001e26, lw) #Global Moran I
> moran <- localmoran(states4.projected$B01001e26, lw, zero.policy=TRUE) #Local Moran I for getting p values for each county
>
> moran_df <- data.frame(
+   unique_id = states4.projected$GEOID,
+   B01001e26 = states4.projected$B01001e26,
+   lag = F.lag,
+   Pvalue = moran[,5]
+ )
> moran_df <- moran_df %>% mutate( quadrant =
+   ifelse(B01001e26 > mean(B01001e26, na.rm = TRUE) & lag > mean(lag, na.rm = TRUE), "HH",
+         ifelse(B01001e26 > mean(B01001e26, na.rm = TRUE) & lag <= mean(lag, na.rm = TRUE), "HL",
+         ifelse(B01001e26 <= mean(B01001e26, na.rm = TRUE) & lag <= mean(lag, na.rm = TRUE), "LL", "LH"))))
> joined.tables <- states4.projected %>% left_join(., moran_df, by = c("GEOID" = "unique_id"))
>
> #Dynamic Map creation
> colors <- c("HH" = "red", "HL" = "blue", "LH" = "orange", "LL" = "green")
> pal <- colorFactor(palette = colors, domain = joined.tables$quadrant)
> leaflet(data = joined.tables) %>%
+   addProviderTiles(providers$CartoDB.Positron, group = "CartoDB") %>%
+   addProviderTiles(providers$OpenStreetMap, group = "OpenStreetMap") %>%
+   addProviderTiles(providers$Esri.WorldImagery, group = "Esri Satellite") %>%
+   addPolygons(fillColor = ~pal(quadrant), stroke = TRUE, color = "black", weight = 0.3, fillOpacity = 0.8, popup = ~paste0(
+     NAME$NAME, " P value: ", Pvalue)) %>%
+   addLegend(pal = pal, values = ~quadrant, title = "Zones", position = "bottomright") %>%
+   addLayersControl(baseGroups = c("CartoDB", "OpenStreetMap", "Esri Satellite"), options = layersControlOptions(collapsed = FALSE))
```



Task-3:

For this task I have recreated the map of my choice from Lab 4 with some little changes and incorporating additional features in the dynamic map.

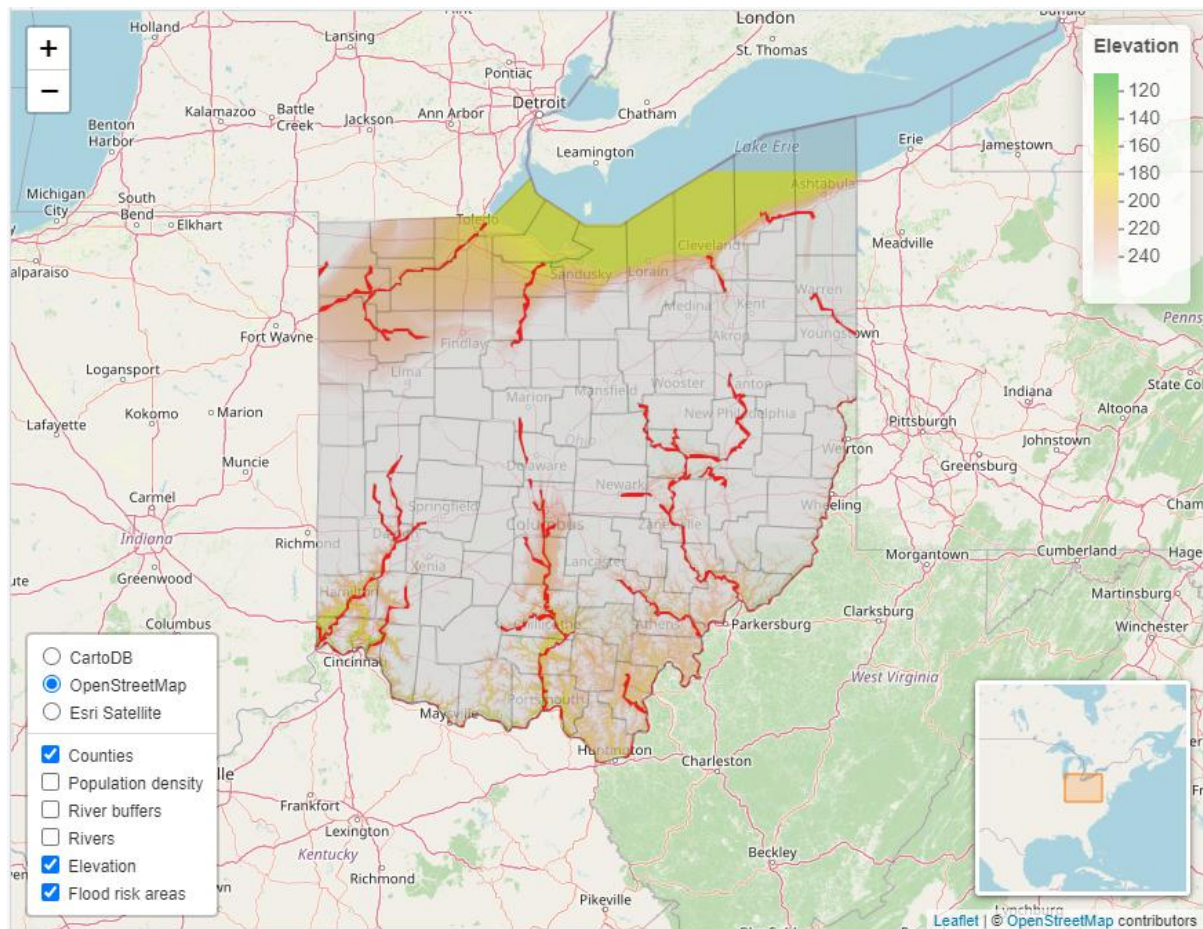
```
> #For this task I am recreating the map of my choice from Lab 4 with different features in the dynamic map
> #data connection
> oh2020 <- read_csv("./data/oh_counties_DP2020.csv")
Rows: 89 Columns: 57
# Column specification
delimter: ",",
chr (2): geoid, name
dbl (55): poptotal, male, mu5, m5to9, m10to14, m15to19, m20to24, m25to29, m30to34, m35to39, m40to44, m45to49, m50to54...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
> counties <- sf::read_sf("./data/oh_counties.gpkg")
> streams.river <- sf::read_sf("./data/oh_rivers.gpkg")
> ohio_dem <- terra::rast("./data/ohio_dem_1.tif")
>
> #data organization
> ohdata <- dplyr::filter(oh2020,name!="ohio")
> joined.table <- left_join(counties, ohdata, by=c("GEOIDFQ"="geoid"))
> ohio <- joined.table %>% mutate(area=st_area(geom))
> ohio <- ohio %>% mutate(pop_dens=(poptotal/as.numeric(area)*1e6))
>
> dem_ohio <- terra::crop(ohio_dem, counties)
> dem_ohio <- terra::mask(ohio_dem, counties)
> river.buffer <- st_buffer(streams.river, dist = 1000) #creating buffer of 1000m from river to identify the flood risk zone
> river.buffer_projected <- river.buffer %>% st_transform(., crs = st_crs(counties))
>
> #identifying flood risk zones and counties with river buffer and low elevation data
> #This part is taking a lot of time to run
> low_elevation <- dem_ohio<50 #identifying the low elevated zones (elevation less than 50)
> low_elev_zones <- as.polygons(low_elevation, dissolve = TRUE)
> low_elev_poly_sf <- st_as_sf(low_elev_zones)
> #This part is taking way too much time to run
> flood_risk_area <- st_intersection(river.buffer_projected, low_elev_poly_sf)
warning message:
attribute variables are assumed to be spatially constant throughout all geometries

> #changing projection for plotting
> ohio <- st_transform(ohio, 4326)
> streams.river <- st_transform(streams.river, 4326)
> river.buffer_projected <- st_transform(streams.river, 4326)
> flood_risk_area <- st_transform(flood_risk_area, 4326)
> dem_ohio <- terra::aggregate(dem_ohio, fact=6)
>
> #Creating dynamic mapping
> pal <- colorNumeric("BuGn", domain = ohio$pop_dens)
> pal2 <- colorNumeric(palette = terrain.colors(25), domain = values(dem_ohio), na.color = "transparent")
>
> leaflet() %>%
+   addProviderTiles(providers$CartoDB.Positron, group = "CartoDB") %>%
+   addProviderTiles(providers$OpenStreetMap, group = "OpenStreetMap") %>%
+   addProviderTiles(providers$Esri.WorldImagery, group = "Esri Satellite") %>%
+   addPolygons(data = ohio, fillColor = "gray", stroke = TRUE,color = "black",
+     weight = 0.3, group="Counties", popup = ~paste0(NAME)) %>%
+   addPolygons(data = ohio, fillColor = ~pal(pop_dens), stroke = TRUE, fillOpacity = 0.9,color = "black",
+     weight = 0.5, group="Population density", popup = ~paste0(NAME)) %>%
+   addPolygons(data = flood_risk_area, fillColor = "red", stroke = TRUE, fillOpacity = 0.9,color = "gray",
+     weight = 0.5, group="Flood risk areas") %>%
+   addPolylines(data = streams.river, color = "blue", weight = 1, opacity = 0.8, group = "Rivers") %>%
+   addPolylines(data = river.buffer_projected, color = "lightblue", weight = 10, opacity = 0.8, group = "River buffers") %>%
+   addRasterImage(dem_ohio, colors = pal2, opacity = 0.7, project = FALSE, group="Elevation") %>%
+   addLegend(pal = pal2, values = values(dem_ohio), title = "Elevation") %>%
+   addLayersControl(baseGroups = c("CartoDB", "OpenStreetMap", "Esri Satellite"), overlayGroups = c("Counties", "Population
density","River buffers", "Rivers", "Elevation","Flood risk areas"),
+     position = "bottomleft", options = layersControlOptions(collapsed = FALSE)) %>%
+   addMiniMap()
```

To avoid the following error and add the raster image in my final dynamic map, I aggregated the raster using facet 6, which I know reduce the accuracy of the raster.

```
Error in addRasterImage_SpatRaster(map = map, x = x, colors = colors, :
Raster image too large; 21705698 bytes is greater than maximum 4194304 bytes
```



QA-1:

This lab provided a valuable learning experience, not only in terms of coding with R but also in developing effective data visualization skills. The best part was that the labs came with lots of challenges which encouraged self-learning and critical thinking in order to handle them. Also, I appreciate the process of leaning from each other through group works. There is no specific thing that I did not like, but sometimes I feel that something can easily be done in ArcGIS.

QA-2:

In my dynamic map, I added the layer on-off options to provide flexibility to my user. The map consists of multiple layers representing different types of spatial data that I have created to make my final map for the lab 4, but in this lab, where I am supposed to develop a dynamic map, it can be useful to the users from different background, such as developers, organization working with flood management. It will enhance the usability and interactivity. Also, as it is a state map, I added a mini map which will help the user to get a better understanding of the location the area during navigation.